

Κ23α - Ανάπτυξη Λογισμικού Για Πληροφοριακά Συστήματα

ΧΕΙΜΕΡΙΝΟ ΕΞΑΜΗΝΟ 2022– 2023

Συντελεστές

ΜΑΡΙΑ ΠΑΡΑΣΚΕΥΟΠΟΥΛΟΥ 1115201800155

ΠΑΣΣΑΚΟΣ ΧΑΤΖΗΟΡΙΔΗΣ ΝΙΚΟΛΑΟΣ 1115201800156

ΚΩΝΣΤΑΝΤΙΝΟΣ ΣΕΙΡΑΣ 1115201800174

Περιεχόμενα

Συντελεστές	1
Περιεχόμενα.....	2
Περιγραφή	3
Δομές Δεδομένων:	4
Templates	4
List	4
Queue.....	4
Vector	4
Εξειδικευμένες δομές:	4
Tuple.....	4
Relation.....	4
Column	4
Histogram	5
Result	5
Άσκηση 1.....	6
Partitioning.....	6
Hoptable - Hopscotch Hashing	8
Neighbourhood.....	9
Άσκηση 2.....	10
Διαδικασία.....	10
Parsing	11
Filtering on constants.....	11
Filtering on equal columns	11
Joins	11
Summation.....	11
Άσκηση 3.....	12
Παραλληλοποίηση	12
Job Scheduler.....	12
Βελτιστοποίηση επερωτήσεων.....	14
Ρυθμίσεις.....	15
Μετρήσεις.....	15

Περιγραφή

Οι πρόσφατες εξελίξεις στην τεχνολογία υπολογιστών οδήγησαν σε σημαντική μείωση του κόστους ανά GB μνήμης RAM, καθώς και σε αύξηση του αριθμού των πυρήνων στις CPU. Οι διακομιστές με TB μνήμης RAM και 20 ή περισσότερους πυρήνες επεξεργασίας είναι πλέον κοινός τόπος. Ως απάντηση, οι σύγχρονες βάσεις δεδομένων σχεδιάζονται για να επωφεληθούν από αυτήν την αυξημένη χωρητικότητα υλικού.

Κατά τη διάρκεια του χειμερινού εξαμήνου του έτους 2022-2023 στα πλαίσια του μαθήματος “Ανάπτυξη Λογισμικού Για Πληροφοριακά Συστήματα” μας ζητήθηκε να κατασκευάσουμε ένα υποσύνολο ενός Συστήματος Βάσεων Δεδομένων που τρέχει εξ ολοκλήρου στη RAM. Το λογισμικό είναι γραμμένο σε C++ χωρίς τη χρήση της STL (Standard Template Library).

Το πρόγραμμα τροφοδοτείται με τα αρχεία σχέσεων και τα αρχεία ερωτημάτων και παράγει τις απαντήσεις στα ερωτήματα στο standard output.

Οι σχέσεις δίνονται σε δυαδική μορφή και έχουν την ακόλουθη δομή:

```
uint64_t numTuples|uint64_t numColumns|uint64_t T0C0|uint64_t
T1C0|..|uint64_t TnC0|uint64_t T0C1|..|uint64_t TnC1|..|uint64_t TnCm
```

Τα ερωτήματα χωρίζονται σε τρία τμήματα. Τις σχέσεις, τα κατηγορήματα και τις προβολές.

Το πρώτο τμήμα ορίζει τις σχέσεις που θα χρησιμοποιηθούν κατά τη ζεύξη.

Το δεύτερο, τις ζεύξεις μεταξύ των σχέσεων αλλά και τα φίλτρα που θα εφαρμοστούν στις στήλες των πινάκων.

Το τρίτο, τις στήλες των πινάκων οι οποίες θα αθροιστούν για να παράγουν το τελικό αποτέλεσμα.

Δίνεται παράδειγμα αντιστοιχίας ενός ερωτήματος σε SQL.

Παράδειγμα:

```
"0 2 4|0.1=1.2&1.0=2.1&0.1>3000|0.0 1.1"
```

Μεταφρασμένο σε SQL:

```
SELECT SUM("0".c0), SUM("1".c1)
FROM r0 "0", r2 "1", r4 "2"
WHERE 0.c1=1.c2 and 1.c0=2.c1 and 0.c1>3000
```

Δομές Δεδομένων:

Templates

Καθώς η χρήση της STL δεν ήταν δυνατή, templates υλοποιήθηκαν για να αντικαταστήσουν συχνά χρησιμοποιούμενες δομές δεδομένων στο προγραμματισμό.

List

Η κλασική υλοποίηση μονά συνδεδεμένης λίστας, καθώς σε λίστες αποθηκεύονται ελάχιστα δεδομένα καθιστώντας τη χρήση πιο σύνθετων λιστών όπως SkipLists και διπλά συνδεδεμένων λιστών μη απαραίτητη.

Queue

Η κλασική υλοποίηση της ουράς, κληρονομεί από την κλάση της λίστας και προσθέτει τις λειτουργίες push και pop που είναι αναμενόμενες.

Vector

Η κλασική υλοποίηση ενός vector, απαραίτητο για το project, καθώς όχι μόνο χρησιμοποιείται στον Parser που δώθηκε, αλλά είναι και εξαιρετικά χρήσιμος για την υλοποίηση άλλων δομών λόγω της ταχύτητας προσπέλασης του.

Εξειδικευμένες δομές:

Tuple

Ένα από ζεύγος τιμών. Χρησιμοποιείται για τα ζεύγη από ids και τιμές.

Relation

Η σχέση (ή πίνακας) που διαβάστηκε από τα δεδομένα αρχεία. Έχει μήκος γραμμών και στηλών και τα δεδομένα τα οποία από τη 2η εργασία και έπειτα είναι αποθηκευμένα χρησιμοποιώντας vectors καθώς ο Parser που δόθηκε εξάγει αυτή τη μορφή. Στην πρώτη εργασία είναι αποθηκευμένα ως διαδοχικές στήλες σε έναν ενιαίο πίνακα. Επίσης στην πρώτη εργασία αυτή η δομή ονομαζόταν Table.

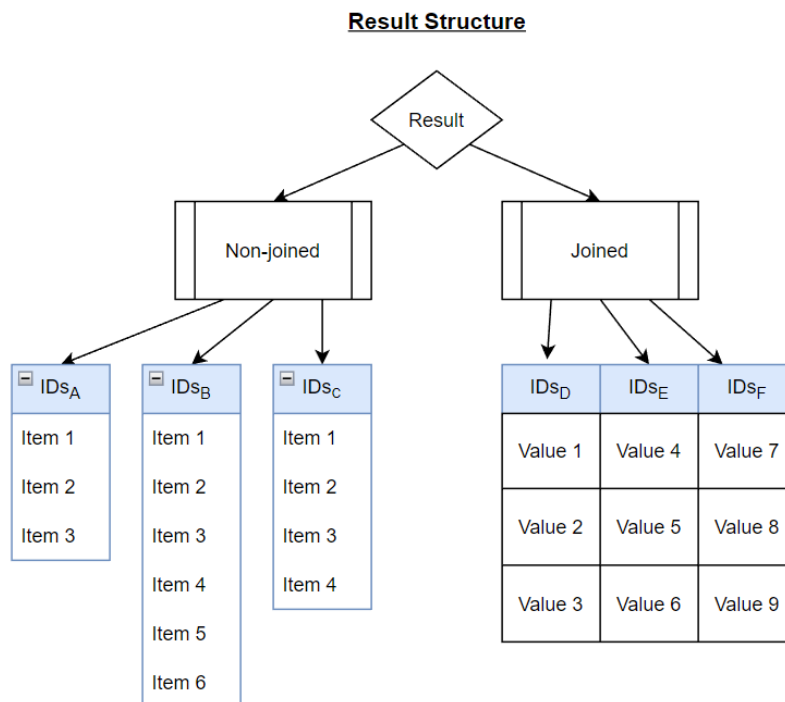
Column

Η Column περιέχει ένα array από Tuples. Χρησιμοποιείται για το ταίριασμα μεταξύ ids του αρχικού πίνακα και των τιμών που συμμετέχουν σε μία πράξη (ζεύξης, ανισότητας κλπ). Στην πρώτη εργασία η δομή ονομαζόταν Relation.

Histogram

Το ιστόγραμμα αποτελείται επίσης από έναν πίνακα από ζεύγη τιμών. Χρησιμοποιείται για την καταμέτρηση πλήθους τιμών (histogram) και την υλοποίηση του αθροιστικού ιστογράμματος (prefix sum)

Result



Η δομή result είναι υπεύθυνη για να κατασκευάζει τις στήλες που θα χρησιμοποιηθούν στα φίλτρα και στις ζεύξεις και για να αποθηκεύει σωστά το ενδιαμέσο αποτέλεσμα.

Το τμήμα των Non-Joined relations περιέχει τις σχέσεις που ακόμα δεν έχουν λάβει μέρος σε κάποια ζεύξη. Αυτό που αξίζει να σημειωθεί είναι πως το non-joined μέρος περιλαμβάνει Vectors ανεξάρτητα μεταξύ τους και διαφορετικών μεγεθών. Σε κάθε relation αντιστοιχεί ένα vector με τα έγκυρα IDs. Τα vectors αυτά αντικαθίστανται με καινούργια καθώς εφαρμόζονται τα φίλτρα.

Το τμήμα των Joined relation περιέχει τα ids των σχέσεων που έχουν λάβει μέρος σε ζεύξη και γι' αυτό το λόγο τα vectors είναι παράλληλα. Αυτό σημαίνει πως έχουν το ίδιο μέγεθος και η ν-οστή τιμή του κάθε vector αντιστοιχεί με τις ν-οστές τιμές των υπόλοιπων, σχηματίζοντας έτσι τη ζεύξη.

Άσκηση 1

Σκοπός της πρώτης φάσης της εργασίας είναι η ζεύξη δύο πινάκων πάνω σε μία στήλη τους. Για την τμηματοποίηση των πινάκων χρησιμοποιείται ο αλγόριθμος Partitioned Hash Join και την τεχνική του hopscotch hashing για την δημιουργία ευρετηρίων.

Partitioning

Η διαδικασία του partitioning είναι έτσι σχεδιασμένη ώστε να μπορεί να κάνει και παραπάνω από 2 passes. Ας υποθέσουμε για χάρη απλότητας ότι έχουμε τον παρακάτω πίνακα R:

Key	Payload
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8

Μετά από το πρώτο pass έστω με $n_1 = 1$ ο πίνακας θα διαμορφωθεί ως εξής:

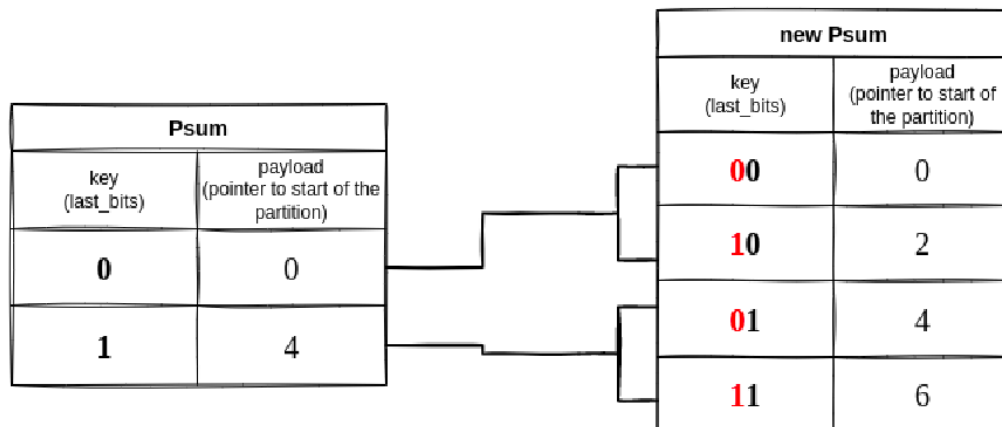
Key	Payload
4	4
8	8
1	1
5	5
2	2
6	6
3	3
7	7

Για να κάνουμε τώρα το partitioning κρατώντας τα αντικείμενα εντός του παλιού τους partition ώστε να εκμεταλλευτούμε το locality, κάνουμε iterate στο κάθε bucket και πρώτα κατασκευάζουμε το νέο prefix sum με τον ακόλουθο τρόπο: Έστω $n_2 = 2$

α: Το παλιό list bits

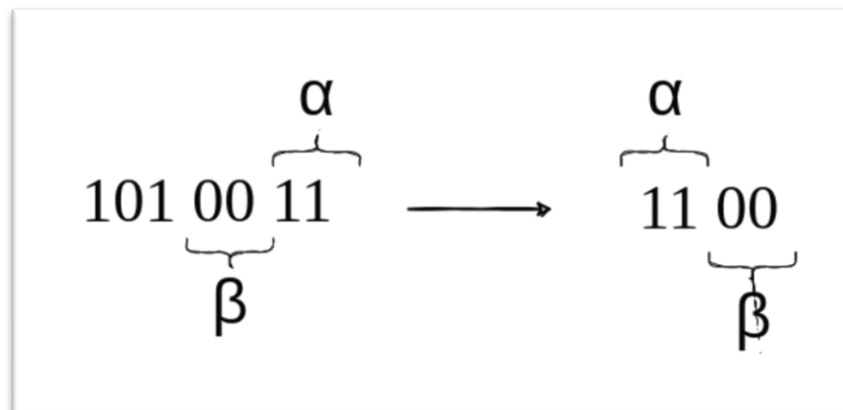
β: για 0 μέχρι τον αριθμό των buckets που δημιουργεί το κάθε ένα bucket, δηλαδή $2^{n_2 - n_1}$

Στη περίπτωση μας το κάθε bucket παράγει 2. Τα νέα buckets θα έχουν τιμές το $\beta\alpha$.

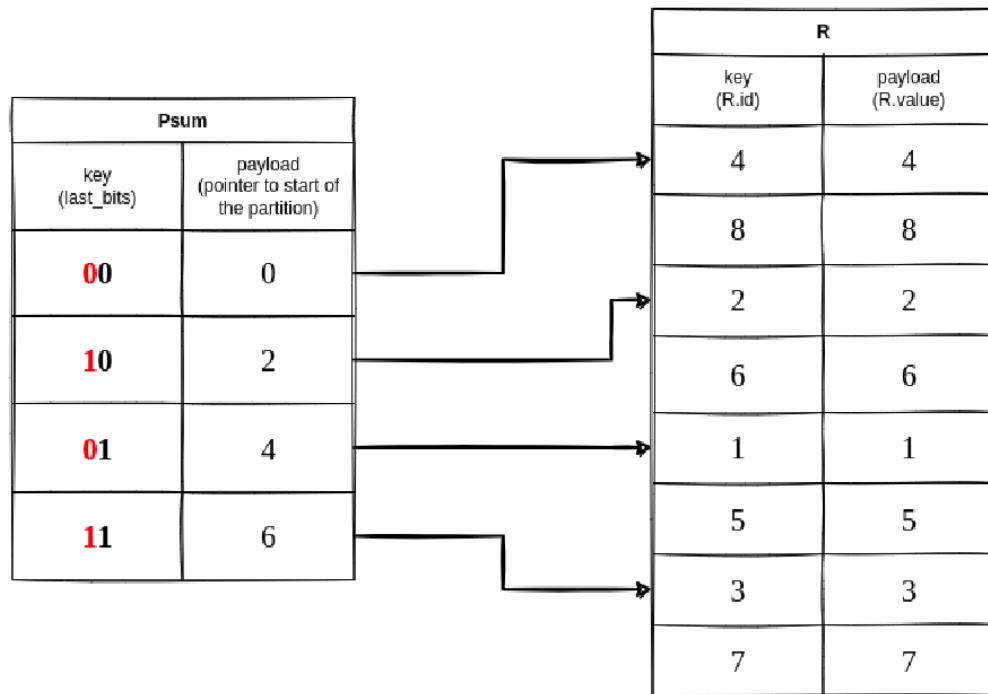


Παρομοίως για να βρούμε το νέο bucket μίας τιμής, θα κρατήσουμε το α του partition στο οποίο βρίσκεται, δηλαδή τα τελευταία n_1 bits και θα εξετάσουμε τα επόμενα $n_2 - n_1$ bits. Για να γίνει πιο σαφές έστω πως έχουμε τον αριθμό 1010011, $n_1 = 2, n_2 = 4$

Τα τελευταία n_1 bits είναι τα 11 και τα 2 επόμενα είναι τα 00. Τώρα αρκεί να τα αντιστρέψουμε για να βρούμε τον αριθμό του καινούργιου bucket:



Στο παράδειγμά μας ο πίνακας αναδιαμορφώνεται ως εξής:



Ο σκοπός του πίνακα bucket_map είναι να διευκολύνει την εύρεση της αρχής ενός bucket και ενώ πιάνει χώρο στη μνήμη, μειώνει την πολυπλοκότητα της εύρεσης του bucket μέσα στο psum από $O(n)$ σε $O(1)$.

Hoptable - Hopscotch Hashing

Η κλάση Hoptable περιλαμβάνει την υλοποίηση του Hashtable που χρησιμοποιεί τον αλγόριθμο Hopscotch hashing. Σε κάθε θέση του hashtable αντιστοιχίζεται ένα Entry που περιλαμβάνει ένα bucket με values και το key αυτών των values και ένα Neighbourhood. Τα buckets και τα Neighbourhoods γίνονται initialize lazily, δηλαδή δεν δημιουργούνται εκ των προτέρων για κάθε θέση του πίνακα, αλλά μόνο σε περίπτωση που πρόκειται να γίνει κάποια εισαγωγή σε αυτά. Το μέγεθος της γειτονιάς γίνεται DEFINE στο header HOODSIZE.hpp, ενώ το αρχικό μέγεθος του hashtable γίνεται define στο Hoptable.hpp. Σε περίπτωση που χρειαστεί να γίνει rehashing, το μέγεθος του hoptable διπλασιάζεται και γίνεται προσπάθεια εισαγωγής των ήδη υπάρχοντων εγγραφών και της επιπλέον εγγραφής. Σε περίπτωση αποτυχίας εισαγωγής των εγγραφών κατά το rehashing γίνεται throw ένα exception και το πρόγραμμα τερματίζει.

Neighbourhood

Η κλάση Neighbourhood είναι abstract (interface) γιατί θεωρήσαμε ότι το implementation μπορεί να παραχθεί και από bitmap και από list ωστόσο στην τρέχουσα φάση έχει δημιουργηθεί μόνο το list implementation (list_neighbourhood). Το list_neighbourhood για την ακρίβεια χρησιμοποιεί Queue implemented από λίστα, διότι εφόσον δεν ζητείται η υλοποίηση της ενέργειας remove, η ουρά φαίνεται να παρέχει την πιο γρήγορη μετακίνηση μιας ελεύθερης θέσης προς την γειτονιά που την χρειάζεται.

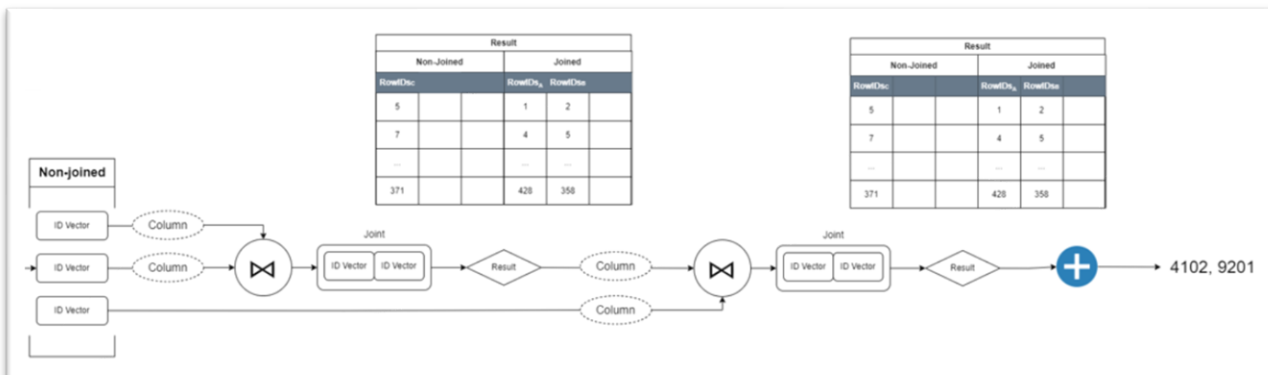
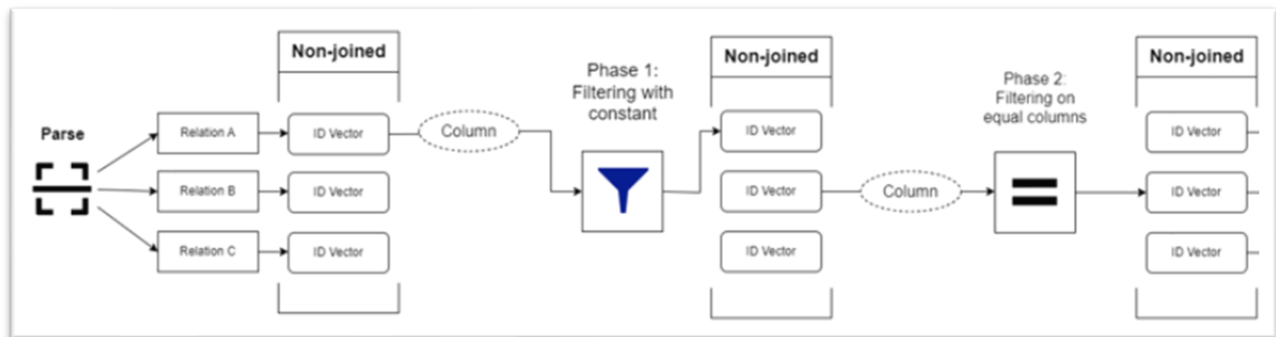
Άσκηση 2

Σκοπός του δεύτερου σκέλους της εργασίας είναι η απάντηση σύνθετων ερωτημάτων που περιλαμβάνουν τις ζεύξεις που υλοποιήθηκαν στο 1ο σκέλος, φίλτρα με σταθερά και αθροίσματα στηλών.

Διαδικασία

Η διαδικασία χωρίζεται στα εξής μέρη:

1. Parsing
2. Filtering on constants
3. Filtering on equal columns
4. Joins
5. Summation



Parsing

Σε αυτή τη φάση για την ανάγνωση των πινάκων και των ερωτημάτων χρησιμοποιείται ο parser που μας δόθηκε από το διαγωνισμό SIGMOD, με τις κατάλληλες αλλαγές, ώστε να χρησιμοποιεί τις δικές μας δομές αντί για της STL. Στο τέλος έχουν δημιουργηθεί τα κατάλληλα relations (πίνακες) και έχουν περαστεί στη δομή Operation Manager που θα αναλάβει να εκτελέσει τα ερωτήματα.

Filtering on constants

Ο Operation Manager δημιουργεί τη δομή Result η οποία περιέχει όλα τα IDs των γραμμών εκείνων που είναι ακόμα έγκυρα σύμφωνα με τα φίλτρα που έχουν εφαρμοστεί από το ερώτημα. Για να γίνει καλύτερα κατανοητή η αρχιτεκτονική Result δείτε στο παράρτημα των δομών δεδομένων.

Για την ώρα χρησιμοποιείται μόνο το μέρος των Non-Joined relations. Σε κάθε relation αντιστοιχεί ένα vector με τα έγκυρα IDs. Τα vectors αυτά αντικαθίστανται με καινούργια καθώς εφαρμόζονται τα φίλτρα. Σε αυτή τη φάση χρησιμοποιείται η συνάρτηση FilterConstant.

Filtering on equal columns

Όπως υποδείχθηκε, όταν γίνεται μία ζεύξη πάνω σε στήλες ίδιου πίνακα αρκεί να ελεγχθεί η ισότητα των τιμών στην ίδια γραμμή. Για αυτό χρησιμοποιείται η συνάρτηση FilterPredicate η οποία όπως στο προηγούμενο στάδιο επιστρέφει μόνο ένα vector με έγκυρα IDs. Τα αποτελέσματα και πάλι αποθηκεύονται στο τμήμα των non-joined relations.

Joins

Σε αυτό το στάδιο είναι που χρησιμοποιείται το τμήμα των joined relations. Αυτό περιέχει Vectors τα οποία είναι ίσου μεγέθους και παράλληλα. Για τη ζεύξη χρησιμοποιείται ένα wrapper class του πρώτου σκέλους της εργασίας το οποίο φτιάχνει ένα joint που περιέχει τα IDs των γραμμών που ταιριάζουν μεταξύ τους σε μορφή ενός vector από tuples. Αυτά τα ζεύγη περνάνε στο Result το οποίο θέτει τα IDs στο joined τμήμα κατάλληλα, με σκοπό να υπάρχει συνέπεια στην αλυσίδα των διαδοχικών joins.

Summation

Τέλος εκτυπώνονται τα αθροίσματα των ζητούμενων στηλών.

Άσκηση 3

Παραλληλοποίηση

Job Scheduler

Ο JobScheduler είναι μια δομή η οποία χρησιμοποιείται για την παράλληλη και ασύγχρονη εκτέλεση εργασιών. Υπάρχουν διάφορα είδη εργασιών που μπορούν να εκχωρηθούν στον JobScheduler για εκτέλεση, η απλούστερη από αυτές είναι η **Job**.

```
#include "jsch.hpp"

JobScheduler jobScheduler(10); /*Specify number of workers*/

/* .. */

auto job = jsch::make_job([&]{..});

jobScheduler.submitJob(job);
```

Συνήθως όμως, θέλουμε οι εργασίες αν και ασύγχρονα να εκτελούνται με μια συγκεκριμένη σειρά. Για εργασίες όπου χρειάζεται η μία να διαδέχεται την άλλη είναι ιδανική η χρήση ενός **JobSequence**.

```
#include "jsch.hpp"

/* ... */

auto job1 = jsch::make_job([&]{...});
auto job2 = jsch::make_job([=]{...});
auto job3 = jsch::make_job([=]() mutable {...});

JobSequence* jobSequence = jobScheduler.make_job_sequence();

jobSequence.then(job1);
jobSequence.then(job2);
jobSequence.then(job3);
```

Υπάρχουν όμως περιπτώσεις όπου η ολοκλήρωση ενός συνόλου εργασιών θέλουμε να προηγείται της έναρξης ενός άλλου συνόλου εργασιών. Σε αυτές τις περιπτώσεις μπορεί να χρησιμοποιηθεί ένα **JobPlan**.

```
#include "jsch.hpp"

JobScheduler jobScheduler(10);

JobPlan* jobPlan = jobScheduler.makeJobPlan();

auto req1 = jsch::make_job([&]{...});
auto req2 = jsch::make_job([&]{...});

jobPlan.addRequiredJob(req1);
jobPlan.addRequiredJob(req2);

auto dep1 = jsch::make_job([&]{...});
auto dep2 = jsch::make_job([&]{...});

jobPlan.addDependent(dep1);
jobPlan.addDependent(dep2);

jobScheduler.submitJob(jobPlan);
```

Όλα αυτά τα είδη εργασιών μπορούν να συνδυαστούν μεταξύ τους για πιο σύνθετες σειρές εκτέλεσης εργασιών.

Ένα σημαντικό ζήτημα είναι ο συγχρονισμός του thread που υποβάλει τις εργασίες στον jobScheduler με αυτές. Για τον λόγο αυτό υπάρχουν τα futures (Η ονομασία προέρχεται από μια βιβλιοθήκη ασύγχρονου προγραμματισμού της javascript αν και δεν είμαι σίγουρος ότι επιτελεί την ίδια λειτουργία.). Ένα future είναι ένα αντικείμενο το οποίο αντιστοιχίζεται σε κάποια δουλειά και μπορεί να χρησιμοποιηθεί για να μπλοκάρει ένα thread μέχρις ότου η δουλειά αυτή να έχει ολοκληρωθεί. Η δημιουργία των futures γίνεται μόνο σε περίπτωση που ένα job έχει υποβληθεί με τη μέθοδο submitWithFuture.

```
#include "jsch.hpp"

int main(){

    JobScheduler jobScheduler(10);

    /*...*/

    Job* job = jsch::make_job( [&]{...});

    Future* future = jobScheduler.submitWithFuture(job);

    future->wait() /* Once "wait" returns, using "wait" on the same future will cause undefined
behaviour */

    /*Calling "wait" is necessary because the program might finish without the job having been executed */

}
```

Βελτιστοποίηση ερωτήσεων

Για την βελτιστοποίηση της σειράς των ερωτημάτων έχει υλοποιηθεί η κλάση **Optimizer**. Πριν την εκτέλεση των ερωτημάτων, ο βελτιστοποιητής υπολογίζει γενικά στατιστικά για κάθε σχέση και έπειτα για κάθε μία και κάθε ερώτημα ξεχωριστά. Για κάθε στήλη είναι ανάγκη να υπολογιστούν οι μετρικές: Ελάχιστο (**lowerBound**), Μέγιστο (**upperBound**), πλήθος δεδομένων (**totalValues**) και μοναδικές τιμές (**distinctValues**). Αυτές θα χρησιμοποιηθούν για τον υπολογισμό του κόστους της κάθε ζεύξης και αποθηκεύονται σε μία δομή **ColumnStatistics**. Μία σχέση αποτελείται από πολλές στήλες. Για αυτό το λόγο χρησιμοποιείται η δομή **RelationStatistics** η οποία εκτός από ένα **Vector** με **ColumnStatistics**, περιέχει έναν δείκτη στην αντίστοιχη σχέση και το πλήθος γραμμών και στηλών.

Ο **Optimizer** κρατάει δείκτες για κάθε σχέση, για κάθε ερώτημα, για κάθε μία **RelationStatistics** δομή και δημιουργεί ξεχωριστά **RelationStatistics** για κάθε σχέση και κάθε ερώτημα. Για να το κάνει αυτό χρησιμοποιεί τις συναρτήσεις **OptimizeFilterEqual**, **OptimizeFilterLessGreater**, **OptimizeSelfJoin**, **OptimizeJoin** αλλάζοντας τα στατιστικά για κάθε στήλη σύμφωνα με τις οδηγίες της εκφώνησης. Για κάθε υποσύνολο των σχέσεων λαμβάνονται υπόψιν όλες οι ζεύξεις που δημιουργούν το συγκεκριμένο αποτέλεσμα. Για παράδειγμα αν ένα αποτέλεσμα προκύπτει μετά από πολλές ζεύξεις ανάμεσα σε στήλες δύο πινάκων, θα επιλεγθεί ο πιο αποτελεσματικός τρόπος να γίνουν αυτές οι ζεύξεις.

Η δομή **BestTree** είναι υπεύθυνη για να κρατάει την καλύτερη σειρά για να γίνει η ζεύξη με κάποιες δεδομένες σχέσεις. Υλοποιεί ένα είδος **Hash Table** το οποίο κατακερματίζει ένα **Vector** με **id** σχέσεων και αποθηκεύει στην αντίστοιχη θέση την καλύτερη στοίχιση των ερωτημάτων ζεύξης. Η σειρά των **ids** κατά τον κατακερματισμό δεν έχει σημασία.

Ρυθμίσεις

Στο αρχείο *include/config.h* μπορούν να ρυθμιστούν οι παρακάτω παράμετροι:

- **L2_CACHE_BYTES:** Το μέγεθος της L2 Cache σε bytes.
- **PASS1_BITS, PASS2_BITS:** Ο αριθμός των bits με τον οποίο θα γίνει το partitioning.
- **THREADS:** Ο αριθμός των νημάτων διαθέσιμα.

Μετρήσεις

Οι μετρήσεις έγιναν σε AMD® Ryzen 5 1600 six-core processor × 12

Multithreading	Optimizer	Runs	Time (μ/s)	MaxHeap (MB)	Workload
✓	✓	100	1855979	252.1	small
✓	✓	10	55759270	5838.0	public
✓	✗	100	1867975	251.7	small
✓	✗	10	64572917	7324.0	public